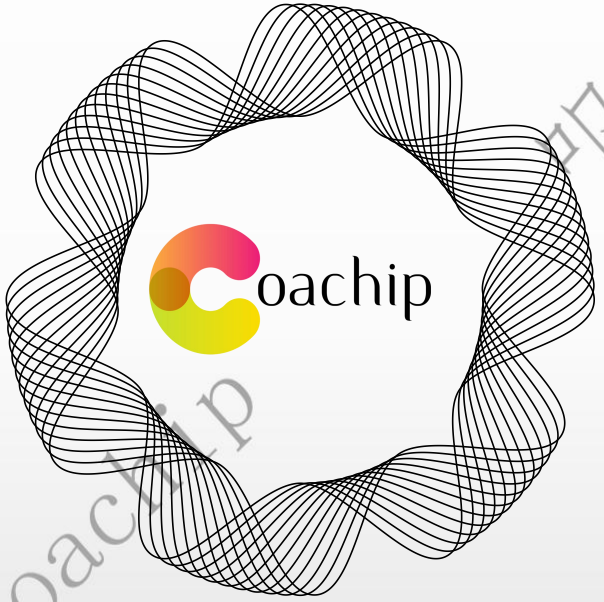


SV交叉覆盖率及覆盖选项

叩特电子Coachip

课程内容及目标

课程内容：

- 交叉覆盖率
- 排除部分cross bin
- 精细化的交叉覆盖率
- 单个实例的覆盖率
- 注释
- 覆盖次数限定
- 覆盖率目标
- covergroup方法

叩特电子Coachip

System Verilog交叉覆盖率—概述

- coverpoint是记录单个变量或者表达式的观测值。
- 如果像记录在某一时刻，**多个变量**之间值的**组合**情况，需要使用交叉（cross）覆盖率。
- cross语句只允许带coverpoint或者简单的变量名。

```
class  Transaction;
    rand   bit [3:0] kind;
    rand   bit [2:0] port;
endclass
Transaction   tr;
covergroup   CovPort;
    kind: coverpoint tr.kind;
    port: coverpoint tr.port;
    cross kind, port;
endgroup
```

叩特电子Coachip

排除部分cross bin

通过使用**ignore_bins**、**binsof**和**intersect**分别指定coverpoint和值域，这样可以清除很多不关心的cross bin。

```
covergroup   Covport;
    port: coverpoint   tr.port
        {bins port[] = {[0:$]};
    }
    kind: coverpoint   tr.kind {
        bins zero = {0};
        bins lo = {[1:3]};
        bins hi[] = {[8:$]};
        bins misc = default;
  }
```

```
cross   kind, port {
      ignore_bins hi = binsof(port) intersect {7};
      ignore_bins md = binsof(port) intersect {0}&&
                  binsof(kind) intersect {[9:11]};
      ignore_bins lo = binsof(kind.lo);
    }
endgroup
```

叩特电子Coachip

精细化的交叉覆盖率

•随着cross覆盖率越来越精细，需要花费不少的时间来指定哪些bin应该被使用或者被忽略。

•更适合的方式是不使用自动分配的cross bin，**自己声明**感兴趣的cross bin。

•假如有两个随机变量a和b，它们带着三种感兴趣的状态，{a==0, b==0}、{a==1, b==0}和{b==1}。

```
class   Transaction;
                    rand bit a, b;
endclass

covergroup   CrossBinNames;
     a: coverpoint tr.a
      { bins a0 = {0};
        bins a1 = {1};
        type_option.weight=0;} // 不计算覆盖率
     b: coverpoint tr.b
      { bins b0 = {0};
        bins b1 = {1};
        type_option.weight=0;} // 不计算覆盖率
     ab: cross a, b
   { bins a0b0 = binsof(a.a0) && binsof(b.b0);
     bins a1b0 = binsof(a.a1) && binsof(b.b0);
     bins b1 = binsof(b.b1); }
   endgroup
```

精细化的交叉覆盖率

```
class   Transaction;
    rand bit a, b;
endclass


covergroup  CrossBinsofIntersect;
    a: coverpoint tr.a
      { type_option.weight=0; } // Don't count this coverpoint
    b: coverpoint tr.b
      { type_option.weight=0; } // Don't count this coverpoint
    ab: cross a,b
  { bins a0b0 = binsof(a) intersect {0} && binsof(b) intersect {0};
    bins a1b0 = binsof(a) intersect {1} && binsof(b) intersect {0};
    bins b1 = binsof(b) intersect {1}; }
endgroup
```

叩特电子Coachip

单个实例的覆盖率

- 如果对一个covergroup**例化**多次，那么默认情况下SV会将所有实例的覆盖率合并到一起。如果需要单独列出每个covergroup实例的覆盖率，则需要设置覆盖选项。

```
covergroup   CoverLength;
    coverpoint   tr.length;
    option.per_instance = 1;
endgroup
```

叩特电子Coachip

注释

- 如果有多个covergroup实例，可以通过参数来对每一个实例传入单独的**注释**。这些注释最终会显示在覆盖率数据的**总结报告**中。

```
covergroup   CoverPort(int lo,hi, string comment);
      option.comment = comment;
      option.per_instance = 1;
      coverpoint port
        {bins range = {[lo:hi]};}
endgroup
...
CoverPort cp_lo = new(0,3, "Low port numbers");
CoverPort cp_hi = new(4,7, "High port numbers");
```

叩特电子Coachip

覆盖次数限定

- 默认情况下，数值采样了1次就可以计入有效的bin。可以通过修改**at_least**来修改每个bin的数值**最少的采样次数**，如果低于at_least数值，则不会被计入bin中。
- option.at_least可以在covergroup中声明来影响所有的coverpoint，也可以在coverpoint中声明来只影响该coverpoint下所有的bin。

即特电子Coachip

覆盖率目标

- 一个covergroup或者一个coverpoint的目标是100%覆盖率。
- 不过你也可以将其设置为低于100%的**目标**。这个选项只会影响覆盖率报告。

```
covergroup  CoverPort;
    coverpoint port;
    option.goal = 90;
endgroup
```

叩特电子Coachip

covergroup方法

- sample()：采样。
- get_coverage()/get_inst_coverage()：获取覆盖率，返回0-100的real数值。
- set_inst_name(string):设置covergroup的名称。
- start()/stop():使能或者关闭覆盖率的收集。

叩特电子Coachip

System Verilog的覆盖率分析

- 使用**$get_coverage()**可以得到总体的覆盖率。
- 也可以使用**covergroup_inst.get_inst_coverage()**来获取单个covergroup实例的覆盖率。
- 这些函数最实际的用处是在一个测试当中**监测**覆盖率的变化。
- 如果覆盖率水平在一段时间之后**没有提高**，那么这个测试就应该**停止**。
- 重启**新的随机种子**或者**测试**可能有望提高覆盖率。
- 如果测试可以**基于功能覆盖率**采取一些深入的行动，例如**重新限定**随机的约束，那将是一件非常好的事情，但是这种测试很难编写。

叩特电子Coachip

练习

添加bins，并收集coverage信息，查看bins的收集情况

TIPS：

makefile elab的时候要添加覆盖率选项：

-cm tgl+branch

run中也要添加：

-cm tgl+branch -cov_dir top_cov

查看报告:

dve -full64 -cov -dir top_cov/top.simv.vdb

```
class Transaction;
    rand logic [31:0] data;
    rand logic [ 2:0] port;
endclass

class Transactor;
    Transaction tr;
    mailbox mbx_in;
    covergroup CovPort;
      port: coverpoint tr.port {
          bins zero = {0};
          ...
      }
      data: coverpoint tr.data {
          bins zero = {0};
          ...
      }
          cross port,data;
    endgroup
  function new(mailbox mbx_in, virtual busifc ifc );
      CovPort = new();
      this.mbx_in = mbx_in;
      this.ifc = ifc;
  endfunction

  task main;
    begin
      mbx_in.get(tr);
      ifc.port <= tr.port;
      ifc.data <= tr.data;
      CovPort.sample();
    end
  endtask
endclass
```

练习

```
program automatic test(busifc ifc);

  initial begin
    Transaction tr;
    mailbox mbx_in;
    Transactor tr_c;

    tr = new();
    mbx_in = new();
    tr_c = new(mbx_in,ifc);

    repeat (32) begin          // Run a few cycles
      assert(tr.randomize);    // Create a transaction
      mbx_in.put(tr);
      tr_c.main(); // send driver and gather coverage
      @ifc.clk;    // Wait a cycle
    end
  end
endprogram
```

叩特电子Coachip

Q&A

昂特电子Coachip